
Sharper Boundaries: Position-Aware CRF Emissions and an Embedding Adapter for Predicting Proteolytic Peptides and Propeptides

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Proteolysis cuts precursor proteins into mature peptides and propeptides; predict-
2 ing where is a sequence segmentation task with direct applications in proteomics,
3 vaccine design, and disease research. The strongest existing approach couples
4 frozen protein language model (pLM) embeddings with a CNN-BiLSTM encoder
5 and an extended-state linear-chain conditional random field (CRF) decoder, but
6 its CRF emissions are identical for every state of a given segment type at a given
7 position, even though the decoder’s own state structure already distinguishes a
8 segment’s start, interior, and end. We close this gap with two small, indepen-
9 dent additions: a zero-initialized boundary head that adds position-specific cor-
10 rections to the relevant CRF emissions, and a lightweight adapter that re-projects
11 the pLM embedding before the encoder. Evaluated with the same 5×4 nested
12 cross-validation DeepPeptide itself uses, on a rebuilt UniProtKB/Swiss-Prot 2026
13 dataset, both additions outperform the base architecture on ESM-2 embeddings
14 individually ($+0.021$ and $+0.026$ F1) and combine to a $+0.054$ F1 gain over the
15 0.576 ± 0.029 base, slightly more than the sum of the two. On higher-capacity
16 ESM-C 6B embeddings the base architecture is statistically indistinguishable from
17 its ESM-2 counterpart under the same protocol (0.588 ± 0.016), yet the same two
18 additions are worth about twice as much there and reach 0.666 ± 0.018 : capac-
19 ity the base decoder cannot use is capacity the additions can. A larger set of
20 exploratory ideas from earlier development is reported in the appendix.

1 Introduction

22 Proteomic databases record intact sequences, not what proteases leave behind, so predicting the
23 products of cleavage has direct value in several applied fields. In vaccine design the same prediction
24 is useful at the design stage, to anticipate which fragments a multi-epitope construct will yield in
25 vivo and to flag segments prone to unwanted cleavage. In disease research, proteolysis is disrupted
26 in neurodegenerative, oncological and endocrine conditions, and a model of its expected and
27 observed cleavage patterns be compared between healthy and diseased samples.

28 The benchmark for this task rewards detection, not localization. A predicted segment counts as
29 correct if both of its endpoints fall within ± 3 residues of a ground-truth segment of the same type,
30 and a single F1 score at this tolerance is reported. Segments are only 5–50 residues long, so this
31 tolerance is wide: on a short peptide it is comparable to the length of the segment itself. A model
32 can therefore raise its F1 simply by finding more segments, even if its boundaries remain imprecise.
33 Conversely, a model can place boundaries much more precisely and gain almost nothing, because
34 predictions within ± 3 residues were already counted as correct. The second case is our concern: the
35 metric cannot see the improvement we are after.

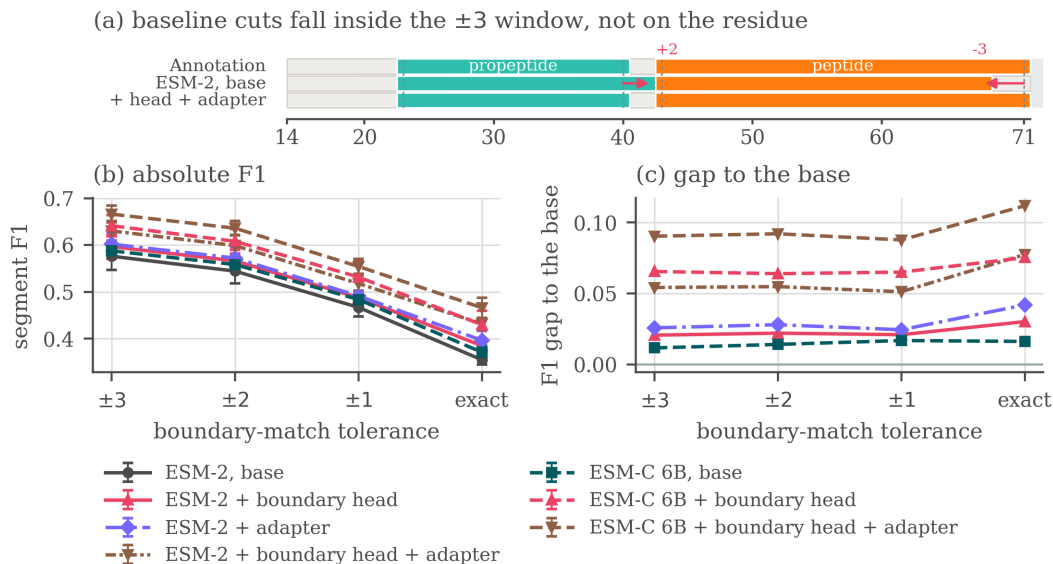


Figure 1: Boundary localization. (a) One precursor from the held-out fold of a nested-CV cell (drawn from residue 14 to its C-terminus), with the ± 3 acceptance window shaded around each annotated cleavage site and the baseline’s displacement in residues marked above it. Both of the baseline’s cuts fall inside that window, so at the headline tolerance its segments count as found, while at an exact match they do not; the model with both additions places them on the annotated residue. (b) Segment F1 as the tolerance tightens from ± 3 to an exact match. (c) The absolute gap to the ESM-2 base at each tolerance; a flat line would mean the curve is a parallel translation of the baseline, and none of them is flat. Corrected matcher throughout; error bars are the standard deviation across the five outer folds (Section 4.3).

We therefore make boundary localization the target of both the model and the evaluation. Our baseline is DeepPeptide [Teufel et al., 2023b], the strongest general-purpose model for this formulation. It has three parts: a frozen protein language model, a CNN-BiLSTM encoder, and a linear-chain CRF. The CRF has separate states for the start, the interior, and the end of a segment, so any valid path must visit them in that order. The decoder is therefore boundary-aware by design. The encoder, however, gives it nothing to work with: every state of the same segment type receives the same emission score. This gap is easy to close, and closing it is the natural way to sharpen boundaries rather than merely find more segments. We close it with two small, independent additions and evaluate them under 5×4 nested cross-validation on the DeepPeptide dataset rebuilt from the 2026 UniProtKB/Swiss-Prot release (Sections 4.1 and 4.3); Appendix D records the single-split protocol used during development and the instability that retired it.

Our contributions are:

- A zero-initialized **boundary head** that supplies the CRF with the position-specific emissions its own state graph already distinguishes, at a cost of 4,678 parameters.
- A lightweight **input adapter** that re-projects frozen pLM embeddings before the encoder, evaluated alone and jointly with the head.
- Gains of +0.021, +0.026 and +0.054 F1 under nested cross-validation, decomposed into their parts: the head buys precision, the adapter buys recall, and both place segment boundaries more precisely, measured on the segments a variant and the base model both find.
- A quantified account of **why this benchmark resists a cleaner protocol**: homology-aware folds are not exchangeable, their spread is as large as either addition measured on them, and averaging it away costs roughly 200 GPU-hours per configuration.

58 2 Related work

59 Most predictors of proteolytic cleavage are specific by construction: one line covers a fixed set of
60 proteases with known recognition motifs, one enzyme or family at a time [Duckert et al., 2004, Song
61 et al., 2012, Li et al., 2023, 2020a,b], a second targets one product class in one kind of organism, most
62 often neuropeptides, and scores cleavage only near basic residues [Southey et al., 2006, Wang et al.,
63 2024]. Neither answers what an organism’s full complement of proteases does to a given precursor.
64 General-purpose models train a compact head on frozen protein-language-model embeddings [Lin
65 et al., 2023, ESM Team, 2024], but they mostly score peptides that have already been excised [Du
66 et al., 2024, Zhu et al., 2025] rather than locating them; the pre-pLM PeptideLocator [Mooney
67 et al., 2013] returns a per-residue similarity heatmap, not a segmentation. DeepPeptide [Teufel
68 et al., 2023b] is, to our knowledge, the only model that segments a precursor into typed peptide
69 and propeptide spans, and it also established the homology-partitioned benchmark used here [Teufel
70 et al., 2023a], so we take both its architecture and its data pipeline as our starting point. Appendix A
71 reviews this literature in more detail.

72 3 Method

73 **The base architecture.** DeepPeptide passes a frozen pLM embedding through a CNN–
74 BiLSTM stack and decodes it with a linear-chain CRF whose state set expands each of
75 {Peptide, Propeptide} into a chain of up to 50 position states, 101 in all, so that a legal path can
76 only realize a contiguous segment of admissible length and the model is trained to produce segments
77 rather than independent residue labels. The encoder, however, computes one emission per label and
78 shares it across all 50 states of that label: the decoder is boundary-aware, the features feeding it are
79 not. The two additions close that gap from opposite ends of the pipeline (Figure 2).

80 **Boundary head.** A small feed-forward block (LayerNorm, Linear, GELU, Linear, hidden size 64)
81 is applied to the contextual per-residue features. For each position and each segment type it emits
82 three numbers, scoring how much that position looks like a segment start, an interior position, or a
83 segment end, and these are added to the emissions of the corresponding position-fixed CRF states.
84 The final linear layer is initialized to zero, so training begins at exactly the base model and the head
85 only learns corrections that lower the loss. It costs 4,678 parameters on top of a trainable stack of
86 224,710, next to a frozen pLM of 650 million.

87 **Adapter.** The second addition leaves the decoder alone and acts on the input. Before the per-
88 residue embedding reaches the CNN–BiLSTM it passes through LayerNorm, Linear, GELU, which
89 re-projects it and reduces its width (1280 to 256 for ESM-2). A pLM embedding is trained on
90 masked-residue recovery, not on cleavage-site localization, and its feature distribution suits neither
91 this task nor the small head that consumes it; a trainable re-projection in front of a frozen encoder is
92 the standard remedy. It is the more expensive of the two additions: the projection itself holds 331,008
93 parameters, partly offset because the convolutions downstream now read 256 channels instead of
94 1280, for a net +232,704.

95 The two additions intervene at different points, so we test them separately and together, giving the
96 2×2 factorial in Table 1. Code, the eight run configurations and a synthetic smoke test are available
97 at <https://anonymous.4open.science/r/cleavage-site-segmentation-5851>.

98 4 Data and evaluation

99 4.1 Dataset

100 A precursor $x = (a_1, \dots, a_L)$ is labelled per residue with $y_t \in \{\text{None}, \text{Peptide}, \text{Propeptide}\}$,
101 and contiguous runs of a label form typed segments. The task is to recover those segments: which
102 stretches of the precursor become mature peptides after cleavage, and which become propeptides
103 that are excised and discarded.

104 DeepPeptide built its dataset from PEPTIDE and PROPEP annotations in the 2022 Swiss-Prot release.
105 We rebuilt it with the same pipeline on the 2026 release [UniProt Consortium, 2025]. The collec-
106 tion grows from 8,449 proteins to 9,619, of which 1,178 were absent in 2022, and 8,897 of them

107 carry ESM-2 embeddings and enter the five folds used here. Folds come from GraphPart [Teufel
108 et al., 2023a] at a 30% pairwise-identity ceiling, balanced by cleavage-motif class. Segment-length
109 filtering, motif balancing and the full composition of the rebuild are given in Appendix B.

110 4.2 Evaluation criterion

111 Peptides and propeptides are matched separately. A prediction is a true positive when its start and
112 its end both fall within $\pm\tau$ residues of a true segment of the same type; unmatched predictions are
113 false positives, unmatched true segments false negatives. We keep $\tau = 3$ as the headline setting,
114 following the original evaluation: annotated cleavage sites carry a few residues of experimental
115 uncertainty, so a prediction that close is practically an exact hit. Sweeping τ down to zero then asks
116 a second question, not whether a peptide was found but how precisely its ends were placed.

117 The reference implementation of this criterion carries an upstream variable-shadowing bug that
118 marks the wrong true segment as matched; everything reported here is re-scored with a corrected
119 matcher (Appendix C).

120 **Agreement with the published baseline.** DeepPeptide reports precision 0.68 and recall 0.49 at
121 ± 3 under nested cross-validation on the 2022 data, an implied F1 of 0.570. We ran the base archi-
122 tecture on that same 2022 release under the same 5×4 protocol, which isolates the release from
123 every other difference: it scores 0.574 ± 0.069 with the original matcher, directly comparable to the
124 published figure, and 0.590 ± 0.064 with the corrected matcher of Appendix C, which is the number
125 comparable to our tables and is higher for the reason given there. On the 2026 rebuild the same
126 architecture reaches 0.573 ± 0.025 (Table 1), so neither the rebuild nor our reimplement moved
127 the baseline.

128 4.3 Nested cross-validation and its cost

129 We evaluate every configuration with the 5×4 nested cross-validation DeepPeptide itself used. For
130 each of five outer folds o , four models are trained on three of the remaining folds and validated
131 on the fourth, then tested on o , which they never saw. An outer fold’s score is the mean over its
132 four inner models and the estimate is the mean over the five outer folds. The reported spread is the
133 standard deviation across outer folds, not across all 20 cells, since four models sharing an outer fold
134 also share a test set and are not independent repeats.

135 **Fold exchangeability.** Homology-aware partitioning is what makes the benchmark meaningful
136 and also what makes the folds unequal. GraphPart keeps homologs together, and in this dataset
137 entire taxa *are* homology clusters, venom families above all. The folds differ in size by a factor of
138 two (1263 to 2572 proteins), and a genus contributing several hundred proteins can be absent from
139 one and dominant in another: of 714 *Conus* sequences, 313 fall in fold 1 and 16 in fold 2, and all
140 293 *Cyriopagopus* avoid fold 0 entirely (Table 2). Motif balancing, which the original pipeline does,
141 does not touch this axis.

142 This puts a floor under how small an effect a single split can resolve. The base architecture scores
143 0.530, 0.572, 0.576, 0.599 and 0.604 on the five outer folds: a range of 0.074, three and a half times
144 the $+0.021$ effect of the boundary head that the same experiment resolves once averaged over folds.
145 Under an earlier single-split protocol we ran during development, that same modification measured
146 as no effect at all, with a confidence interval covering zero (Appendix D). The modification did not
147 change; the resolution did.

148 **The case against a third level of nesting.** The protocol still selects architectures on outer-fold
149 scores, which a sealed design would forbid; closing that needs a third held-out level. One cell costs
150 about 10 GPU-hours, so a configuration of 20 cells costs roughly 200 and the full $2 \times 2 \times 2$ grid
151 roughly 1,600, which is weeks of continuous single-GPU compute for the configurations reported
152 here. A third level multiplies that again while cutting the training fraction from $3/5$ to $2/5$ of an
153 already small dataset. At this size the protocol cannot be made both clean and affordable, so we
154 state the trade-off rather than leave it implicit.

Table 1: 5×4 nested cross-validation with the corrected matcher, mean $\pm$ std. over the five outer folds. Precision and recall are at the headline ± 3 tolerance; the four F1 columns tighten the boundary-match tolerance to an exact residue. *Growth* is how much a row’s F1 advantage over the ESM-2 base widens between ± 3 and an exact match, so a value near zero would mean its tolerance curve is a parallel translation of the baseline.

Boundary	Adapter	at ± 3		F1 by tolerance				growth
		P	R	± 3	± 2	± 1	exact	
<i>ESM-2</i>								
—	—	0.621 ± 0.025	0.539 ± 0.045	0.576 ± 0.029	0.544	0.467	0.354	—
✓	—	0.673 ± 0.029	0.538 ± 0.036	0.597 ± 0.026	0.566	0.488	0.384	+0.010
—	✓	0.628 ± 0.019	0.579 ± 0.040	0.602 ± 0.026	0.572	0.491	0.396	+0.016
✓	✓	0.667 ± 0.032	0.598 ± 0.026	0.630 ± 0.021	0.599	0.518	0.432	+0.023
<i>ESM-C 6B</i>								
—	—	0.620 ± 0.017	0.560 ± 0.027	0.588 ± 0.016	0.558	0.483	0.371	+0.005
✓	—	0.731 ± 0.010	0.572 ± 0.030	0.641 ± 0.022	0.608	0.532	0.429	+0.010
✓	✓	0.690 ± 0.013	0.646 ± 0.033	0.666 ± 0.018	0.636	0.554	0.466	+0.021

5 Results

Table 1 reports the 2×2 factorial on ESM-2 and, on ESM-C 6B, the base architecture with the boundary head and with both additions, all under the protocol of Section 4.3 with the corrected matcher.

The two additions act on different errors. Each improves F1 by a similar amount on its own, and together they give +0.054, slightly more than the sum of the parts. Splitting F1 into its components explains why. The boundary head is almost purely a precision effect: precision rises by 0.052 while recall does not move at all (−0.002). The adapter is mostly a recall effect: recall rises by 0.040 against 0.007 of precision. Applied together they recover both, +0.046 precision and +0.059 recall. They add up because they act on different error types. One suppresses spurious segments; the other finds segments the base model misses.

Tightening the tolerance. Figure 1 and the F1 columns of Table 1 sweep τ from ± 3 to an exact match. Absolute F1 falls steeply for every model: the best configuration goes from 0.630 at ± 3 to 0.432 at the exact residue. Placing a cleavage site exactly is still an open problem.

A model that is better overall retains a different *fraction* of its ± 3 score even when no boundary is placed better, so we compare absolute gaps rather than ratios. Every gap widens as the tolerance tightens, from +0.021 to +0.030 for the boundary head, +0.026 to +0.042 for the adapter and +0.054 to +0.077 for the two together, and each is nearly flat from ± 3 to ± 1 before opening up at the last step: what separates these models is specifically whether they hit the exact residue.

A widening gap is still not proof of better localization, since a model that finds more segments also gains ground at every tolerance. To hold detection fixed we compared each variant against the base on the true segments that *both* localize within ± 3 , matched cell by cell and segment by segment. On that set the share of boundaries placed on the exact residue rises from 0.630 to 0.652 for the boundary head ($+0.022 \pm 0.015$), from 0.619 to 0.668 for the adapter ($+0.049 \pm 0.021$) and from 0.621 to 0.698 for the two together ($+0.074 \pm 0.017$), each positive on all five outer folds; segment counts are in Appendix D.4. A gate is itself a choice, so Appendix E asks the same question without one, scoring every model against all true and all predicted segments under an overlap criterion rather than a tolerance. It answers a question the tolerance cannot: when a hit needs only a single residue of overlap, neither addition gains anything measurable (−0.013 for the head, +0.006 for the adapter), and both gain steadily as the overlap requirement tightens. What they buy is placement rather than coverage, which is why a criterion with a ± 3 window records part of it as recall. Read with the precision/recall split above, this gives each block a job: the head filters spurious segments and sharpens the ones that survive, while the adapter mostly finds segments the base model misses.

Capacity the base decoder cannot use. The base architecture on ESM-C 6B reaches 0.588 ± 0.016 against 0.576 ± 0.029 on ESM-2. The intervals overlap, so an embedding twice as wide buys no confirmed improvement on its own, while either architectural addition on the narrower one

does. The additions, though, are worth about twice as much on it. Measured against its own base and paired by outer fold, the boundary head gains $+0.054 \pm 0.026$ on ESM-C 6B against $+0.021 \pm 0.007$ on ESM-2, and the two together $+0.078 \pm 0.009$ against $+0.054 \pm 0.016$; all four are positive on five folds of five. Read the other way, the same embedding swap is worth $+0.012$ F1 under the base architecture and $+0.036$ under both additions. The two embeddings are covered by slightly different protein sets (Section 4.1), so we checked this again on the tolerance-free suite of Appendix E, which scores every model on the 35,504 protein-cells common to all runs: there the swap is worth $+0.005$ at IoU ≥ 0.5 under the base architecture, on two folds of five, and $+0.024$ under both additions, on five of five. The capacity is in the wider embedding either way; what changes is whether the decoder can reach it.

6 Conclusion

Two small additions to the DeepPeptide recipe, a zero-initialized boundary head at the decoder and a lightweight adapter at the input, improve segment F1 by $+0.054$ together and by $+0.021$ and $+0.026$ on their own. They combine because they act on different errors, the head suppressing spurious segments and the adapter finding more real ones, and both place segment boundaries more precisely, so their advantage widens as the match tolerance tightens.

The evaluation carries a second message: homology-aware partitioning leaves the folds taxonomically non-exchangeable, their spread is as large as either addition measured on them, and averaging it away, at about 200 GPU-hours per configuration, is what makes those effects visible at all.

Limitations. The reported spread covers fold composition only, not seed variation, so the true uncertainty is wider. The adapter alone was not run under nested cross-validation on ESM-C 6B, so the factorial is complete on ESM-2 only. The protocol still selects architectures using outer-fold scores rather than a sealed third level, for the reasons given in Section 4.3. A larger set of ideas from earlier development, including a structural channel, an auxiliary bond loss, a telescopic segment CRF and LoRA fine-tuning, was only ever tested under the weaker protocol and is reported in Appendix D as unconfirmed.

References

- José Juan Almagro Armenteros, Marco Salvatore, Olof Emanuelsson, Ole Winther, Gunnar von Heijne, Arne Elofsson, and Henrik Nielsen. Detecting sequence signals in targeting peptides using deep learning. *Life Science Alliance*, 2(5):e201900429, 2019. doi: 10.26508/lsa.201900429.
- Zhenjiao Du, Xingjian Ding, William Hsu, Arslan Munir, Yixiang Xu, and Yonghui Li. pLM4ACE: a protein language model based predictor for antihypertensive peptide screening. *Food Chemistry*, 431:137162, 2024. doi: 10.1016/j.foodchem.2023.137162.
- Peter Duckert, Søren Brunak, and Nikolaj Blom. Prediction of proprotein convertase cleavage sites. *Protein Engineering, Design and Selection*, 17(1):107–112, 2004. doi: 10.1093/protein/gzh013.
- Ahmed Elnaggar, Hazem Essam, Wafaa Salah-Eldin, Walid Moustafa, Mohamed Elkerdawy, Charlotte Rochereau, and Burkhard Rost. Ankh: Optimized protein language model unlocks general-purpose modelling, 2023. URL <https://arxiv.org/abs/2301.06568>.
- ESM Team. Esm cambrian: Revealing the mysteries of proteins with unsupervised learning. <https://www.evolutionaryscale.ai/blog/esm-cambrian>, December 2024.
- Thomas Hayes, Roshan Rao, Halil Akin, Nicholas J. Sofroniew, Deniz Oktay, Zeming Lin, Robert Verkuil, Vincent Q. Tran, Jonathan Deaton, Marius Wiggert, Rohil Badkundri, Irhum Shafkat, Jun Gong, Alexander Derry, Raul S. Molina, Neil Thomas, Yousuf A. Khan, Chetan Mishra, Carolyn Kim, Liam J. Bartie, Matthew Nemeth, Patrick D. Hsu, Tom Sercu, Salvatore Candido, and Alexander Rives. Simulating 500 million years of evolution with a language model. *Science*, 387(6736):850–858, 2025. doi: 10.1126/science.ads0018.
- Michael Heinzinger, Konstantin Weissenow, Joaquin Gomez Sanchez, Adrian Henkel, Milot Mirdita, Martin Steinegger, and Burkhard Rost. ProstT5: bilingual language model for protein sequence and structure. *NAR Genomics and Bioinformatics*, 6(4):lqae150, 2024. doi: 10.1093/nargab/lqae150.

241 Fuyi Li, Jinxiang Chen, André Leier, Tatiana Marquez-Lago, Quanzhong Liu, Yanze Wang, Jerico
242 Revote, A. Ian Smith, Tatsuya Akutsu, Geoffrey I. Webb, Lukasz Kurgan, and Jiangning Song.
243 DeepCleave: a deep learning predictor for caspase and matrix metalloprotease substrates and
244 cleavage sites. *Bioinformatics*, 36(4):1057–1065, 2020a. doi: 10.1093/bioinformatics/btz721.

245 Fuyi Li, André Leier, Quanzhong Liu, Yanan Wang, Dongxu Xiang, Tatsuya Akutsu, et al.
246 Procleave: predicting protease-specific substrate cleavage sites by combining sequence and
247 structural information. *Genomics, Proteomics & Bioinformatics*, 18(1):52–64, 2020b. doi:
248 10.1016/j.gpb.2019.08.002.

249 Fuyi Li, Cong Wang, Xudong Guo, Tatsuya Akutsu, Geoffrey I. Webb, Lachlan J. M. Coin, Lukasz
250 Kurgan, and Jiangning Song. ProsperousPlus: a one-stop and comprehensive platform for ac-
251 curate protease-specific substrate cleavage prediction and machine-learning model construction.
252 *Briefings in Bioinformatics*, 24(6):bbad372, 2023. doi: 10.1093/bib/bbad372.

253 Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin,
254 Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom
255 Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level
256 protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. doi: 10.1126/
257 science.ade2574.

258 Catherine Mooney, Niall J. Haslam, Gianluca Pollastri, and Denis C. Shields. Towards the im-
259 proved discovery and design of functional peptides: common features of diverse classes permit
260 generalized prediction of bioactivity. *PLoS ONE*, 7(10):e45012, 2012. doi: 10.1371/journal.pone.
261 0045012.

262 Catherine Mooney, Niall J. Haslam, Thérèse A. Holton, Gianluca Pollastri, and Denis C. Shields.
263 PeptideLocator: prediction of bioactive peptides in protein sequences. *Bioinformatics*, 29(9):
264 1120–1126, 2013. doi: 10.1093/bioinformatics/btt103.

265 Alexander Rives, Joshua Meier, Tom Sercu, Siddharth Goyal, Zeming Lin, Jason Liu, Demi Guo,
266 Myle Ott, C. Lawrence Zitnick, Jerry Ma, and Rob Fergus. Biological structure and function
267 emerge from scaling unsupervised learning to 250 million protein sequences. *Proceedings of the*
268 *National Academy of Sciences*, 118(15):e2016239118, 2021.

269 Jiangning Song, Hao Tan, Andrew J. Perry, Tatsuya Akutsu, Geoffrey I. Webb, James C. Whisstock,
270 and Robert N. Pike. PROSPER: an integrated feature-based tool for predicting protease substrate
271 cleavage sites. *PLoS ONE*, 7(11):e50300, 2012. doi: 10.1371/journal.pone.0050300.

272 Jiangning Song et al. PROSPERous: high-throughput prediction of substrate cleavage sites for
273 90 proteases with improved accuracy. *Bioinformatics*, 34(4):684–687, 2018. doi: 10.1093/
274 bioinformatics/btx670.

275 Bruce R. Southey, Andinet Amare, Tyler A. Zimmerman, Sandra L. Rodriguez-Zas, and Jonathan V.
276 Sweedler. NeuroPred: a tool to predict cleavage sites in neuropeptide precursors and provide the
277 masses of the resulting peptides. *Nucleic Acids Research*, 34(Web Server issue):W267–W272,
278 2006. doi: 10.1093/nar/gkl161.

279 Felix Teufel, José Juan Almagro Armenteros, Alexander Rosenberg Johansen, Magnús Halldór Gís-
280 lason, Silas Irby Pihl, Konstantinos D. Tsirigos, Ole Winther, Søren Brunak, Gunnar von Heijne,
281 and Henrik Nielsen. SignalP 6.0 predicts all five types of signal peptides using protein language
282 models. *Nature Biotechnology*, 40(7):1023–1025, 2022. doi: 10.1038/s41587-021-01156-3.

283 Felix Teufel, Magnús Halldór Gíslason, José Juan Almagro Armenteros, Alexander Rosenberg Jo-
284 hansen, Ole Winther, and Henrik Nielsen. GraphPart: homology partitioning for biological se-
285 quence analysis. *NAR Genomics and Bioinformatics*, 5(4):lqad088, 2023a. doi: 10.1093/nargab/
286 lqad088.

287 Felix Teufel, Jan Christian Refsgaard, Christian Toft Madsen, Carsten Stahlhut, Mads Grønborg, Ole
288 Winther, and Dennis Madsen. Deeppeptide predicts cleaved peptides in proteins using conditional
289 random fields. *Bioinformatics*, 39(10):btad616, 10 2023b. ISSN 1367-4811. doi: 10.1093/
290 bioinformatics/btad616. URL <https://doi.org/10.1093/bioinformatics/btad616>.

- 291 UniProt Consortium. UniProt: the universal protein knowledgebase in 2025. *Nucleic Acids Re-*
 292 *search*, 53(D1):D609–D617, 2025. doi: 10.1093/nar/gkae1010. URL [https://academic.oup-](https://academic.oup.com/nar/article/53/D1/D609/7902999)
 293 [com/nar/article/53/D1/D609/7902999](https://academic.oup.com/nar/article/53/D1/D609/7902999).
- 294 Michel van Kempen, Stephanie S. Kim, Charlotte Tumescheit, Milot Mirdita, Jeongjae Lee,
 295 Cameron L. M. Gilchrist, Johannes Söding, and Martin Steinegger. Fast and accurate pro-
 296 tein structure search with Foldseek. *Nature Biotechnology*, 42:243–246, 2024. doi: 10.1038/
 297 s41587-023-01773-0.
- 298 Lei Wang, Chen Huang, Mingxia Wang, Zhidong Xue, and Yan Wang. NeuroPred-PLM: an inter-
 299 pretable and robust model for neuropeptide prediction by protein language model. *Briefings in*
 300 *Bioinformatics*, 24(2):bbad077, 2023. doi: 10.1093/bib/bbad077.
- 301 Lei Wang, Zilu Zeng, Zhidong Xue, and Yan Wang. DeepNeuropePred: a robust and universal tool
 302 to predict cleavage sites from neuropeptide precursors by protein language model. *Computational*
 303 *and Structural Biotechnology Journal*, 23:309–315, 2024. doi: 10.1016/j.csbj.2023.12.004.
- 304 M. Zhang, J. Zhou, X. Wang, et al. DeepBP: ensemble deep learning strategy for bioactive peptide
 305 prediction. *BMC Bioinformatics*, 25:352, 2024. doi: 10.1186/s12859-024-05974-5.
- 306 Lei Zhu, Hao Sun, and Sen Yang. BPFun: a deep learning framework for bioactive peptide function
 307 prediction using multi-label strategy by transformer-driven and sequence rich intrinsic informa-
 308 tion. *BMC Bioinformatics*, 26(1):187, 2025. doi: 10.1186/s12859-025-06190-5.

309 **Impact Statement**

310 This paper improves a computational tool for predicting proteolytic cleavage products from pro-
 311 tein sequence. Potential applications include interpreting proteomics data, informing vaccine and
 312 peptide-drug design, and studying disease-associated proteolysis; we are not aware of societal risks
 313 specific to this work beyond those generally associated with more accurate protein sequence analysis
 314 tools.

Appendix

A Extended review of prior work

Section 2 surveys this literature in the space a workshop paper allows. This appendix gives the same material in the detail a reader unfamiliar with the task will want.

Methods for predicting proteolytic peptides fall into three broad classes.

Protease-specific models. These predict cleavage only for a limited set of enzymes, most of which have a known recognition motif. ProP [Duckert et al., 2004] targets cleavage by the PACE/PC family in animals and plants; PROSPER [Song et al., 2012] and its successor PROSPERous [Song et al., 2018] predict cleavage for one chosen enzyme at a time, as does the later ProsperousPlus [Li et al., 2023]; DeepCleave [Li et al., 2020a] is restricted to caspases and matrix metalloproteinases; and Procleave [Li et al., 2020b] likewise handles one enzyme at a time and additionally requires 3D structural input. The shared limitation of this class is scope: in practice, one is usually interested in the combined effect of all proteases active in an organism, not any single enzyme.

Organism- or peptide-type-specific models. A separate class targets neuropeptides or other bioactive fragments specific to a given organism. NeuroPred [Southey et al., 2006] and the species-agnostic DeepNeuropePred [Wang et al., 2024] only detect cleavage near a handful of basic residues; NeuroPred-PLM [Wang et al., 2023] does not localize cleavage sites at all but classifies an already-excised sequence. A related group of models finds signal subsequences rather than any bioactive peptide: SignalP [Teufel et al., 2022] and TargetP [Almagro Armenteros et al., 2019] identify the type of signal peptide, which indicates where a protein is trafficked.

General-purpose models on pLM embeddings. Protein language models (pLMs) are transformers trained without supervision on large sequence corpora, typically by masked-residue recovery, and they produce transferable per-residue embeddings: ESM-2 [Lin et al., 2023], the earlier ESM line [Rives et al., 2021], ESM Cambrian [ESM Team, 2024, Hayes et al., 2025], and Ankh [El-naggar et al., 2023]. Compact task-specific heads are then trained on top of these frozen embeddings. Besides **DeepPeptide** [Teufel et al., 2023b] itself, this recipe underlies DeepNeuropePred, NeuroPred-PLM, and SignalP 6, as well as models that score already-excised peptides rather than finding them: pLM4ACE [Du et al., 2024] predicts ACE-inhibitory activity, and BPFun [Zhu et al., 2025] and DeepBP [Zhang et al., 2024] predict broader bioactivity, building on earlier pre-pLM work such as PeptideRanker [Mooney et al., 2012]. Before pLMs, PeptideLocator [Mooney et al., 2013] addressed a problem close to ours, but its output is a per-residue heatmap of similarity to known bioactive peptides rather than an explicit segmentation, and it was the main point of comparison in the original DeepPeptide paper.

Baseline architecture: DeepPeptide. Since we build directly on it throughout this paper (Section 3), DeepPeptide’s architecture is worth describing in detail rather than treating as a black box. It first encodes a precursor sequence with a frozen pLM (ESM-2), producing a per-residue embedding that is not fine-tuned during training. This embedding is refined by a convolution, a single-layer bidirectional LSTM and a second convolution. The published description does not fix the widths: they are searched with Optuna inside the inner cross-validation loop. The configuration we train throughout uses 32 convolutional filters of width 3 and an LSTM hidden size of 64 per direction, giving 224,710 trainable parameters for the base model. These features are consumed by a linear-chain CRF decoder whose state set is extended well beyond the three raw labels {None, Peptide, Propeptide}: each of the two positive labels is expanded into a chain of states long enough to represent segments up to a fixed maximum length (101 states in total for the 50-residue cap used here, Section 4.1), so that a legal path through the CRF can only realize a contiguous segment of valid length, with dedicated states marking its first and last positions. This makes segmentation, rather than independent per-residue classification, the object the model is directly optimized for – and it is also where the gap described in Section 1 lives: every state in this extended set that shares a type receives the same emission from the encoder, regardless of whether it marks the start, interior, or end of a segment.

Baseline dataset for this task. Beyond its architecture, DeepPeptide also established the data resource this line of work relies on: precursor sequences from UniProtKB/Swiss-Prot annotated with PEPTIDE and PROPEP feature types, partitioned into homology-aware folds with GraphPart [Teufel et al., 2023a] at a 30% pairwise-identity threshold and additionally balanced by cleavage-flanking motif, then evaluated with the same family of nested cross-validation we adopt (Section 4.3). This

gave the field a standardized, already homology-controlled benchmark rather than an ad hoc collection of previously published peptide lists, and both the curated data and the construction pipeline that produced it are public. Section 4.1 describes what we do with that resource: rebuilding it on a current UniProtKB/Swiss-Prot release so that it has enough scale and coverage for 5×4 nested cross-validation.

Among these, DeepPeptide is, to our knowledge, the only model that poses the task as full segmentation of the precursor into typed peptide and propeptide segments, and it is the strongest available baseline for this formulation. We therefore use its architecture and its dataset-construction methodology as the starting point for this work, rather than treating it as a system to audit: our contribution is an architectural addition to this general recipe (Section 3), evaluated under a more conservative protocol (Section 4) on data rebuilt at greater scale (Section 4.1).

B Task formulation and dataset composition

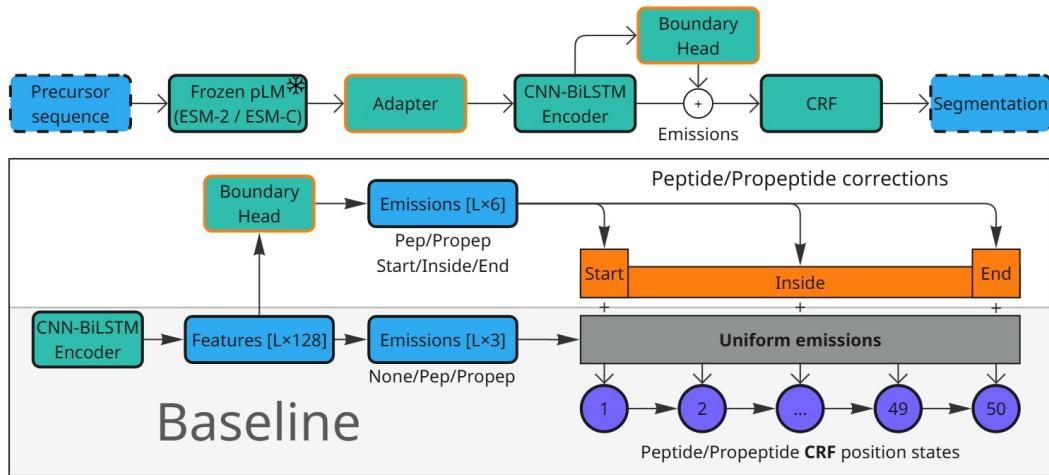


Figure 2: The pipeline from sequence to decoded segments, and where the two additions change it (top), with a zoom into the emission step (bottom). The adapter re-projects the frozen pLM embedding before the CNN-BiLSTM encoder. The boundary head reads the encoder output and adds a position-specific correction to the per-state CRF emissions before decoding. In the baseline (bottom, shaded) a single 3-way emission is repeated across every state of a given type; the boundary head produces a start/inside/end correction per type on top of it, so the first and last states of a segment no longer see the same evidence as its interior.

Table 2: Proteins per outer fold, and how three frequent genera distribute across them. *Cyriopagopus* and *Lycosa* are spider venoms, *Conus* a cone snail; each is a homology cluster that GraphPart is obliged to keep intact. *Homo*, by contrast, is spread evenly (71/81/66/130/84).

	total	f0	f1	f2	f3	f4
All proteins	8897	1558	2572	1263	2025	1479
<i>Conus</i>	714	160	313	16	149	76
<i>Cyriopagopus</i>	293	0	62	45	9	177
<i>Lycosa</i>	163	5	42	10	106	0

Two filters from the original pipeline are kept unchanged. Segments shorter than 5 or longer than 50 residues are dropped: the CRF’s state count fixes the upper bound, and a segment shorter than five residues is barely scoreable at a ± 3 tolerance. Folds are then balanced by cleavage-motif class, obtained by k -means ($k = 50$) on ESM-2 embeddings of the four residues flanking each annotated boundary, on top of the 30% pairwise-identity ceiling GraphPart enforces between folds.

Table 3: Dataset composition: the 2022 release used by the original DeepPeptide against the 2026 release rebuilt here. Counts are before homology partitioning.

	2022	2026
Proteins	8449	9619
of which absent in 2022	—	1178 ($\approx 12\%$)
Peptide segments	6372	7431
Propeptide segments	8211	9140
Median peptide length (residues)	21	20

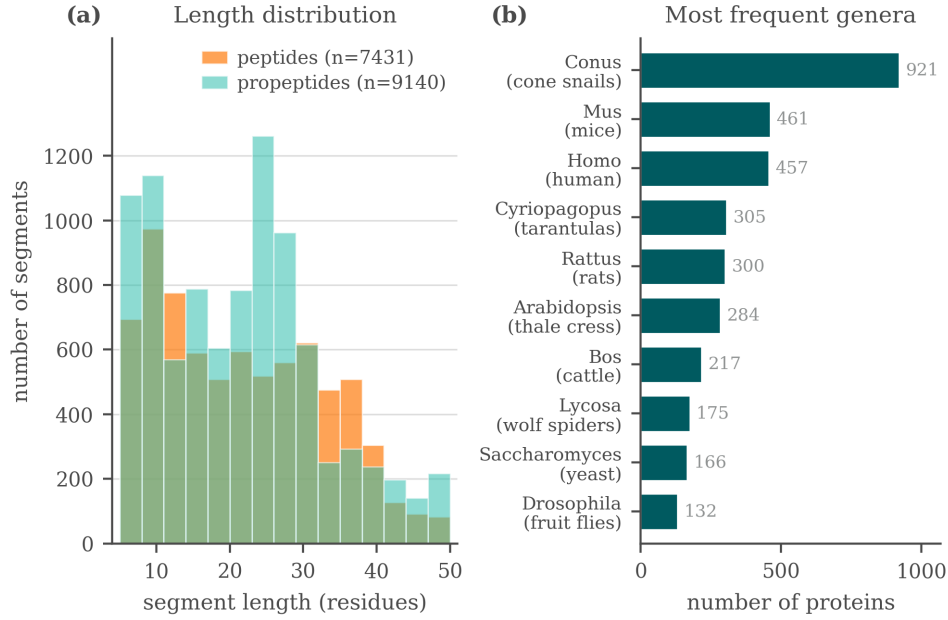


Figure 3: Composition of the rebuilt 2026 dataset: segment length, type, and genus distributions.

386 B.1 Problem formulation

A precursor protein is a sequence of amino acids

$$x = (a_1, a_2, \dots, a_L), \quad a_t \in \Omega,$$

where Ω is the 20-letter amino acid alphabet and L is the sequence length. The task is to predict which contiguous stretches of x become mature **peptides** after cleavage, and which become **propeptides** – auxiliary stretches that are excised during maturation and removed, without themselves being a final active product. Formally, each position is assigned a label

$$y_t \in \{\text{None}, \text{Peptide}, \text{Propeptide}\},$$

and contiguous runs of the same label form typed **segments** (start, end) pairs:

$$\begin{array}{ccccc} \text{MGK} & \text{RALR} & \text{RSGK} & \text{VR} & \text{NL} \\ \underbrace{\hspace{1cm}} & \underbrace{\hspace{1cm}} & \underbrace{\hspace{1cm}} & \underbrace{\hspace{1cm}} & \underbrace{\hspace{1cm}} \\ \text{None} & \text{Propeptide} & \text{Peptide} & \text{None} & \text{Peptide} \end{array}$$

387 How a predicted segment is scored against the ground truth – including the tolerance used for bound-
 388 ary matching – is defined later, in Section 4.2, alongside the rest of the evaluation protocol; it is an
 389 evaluation-time decision rather than part of the task definition itself.

390 C Metric implementation bug

391 While auditing the reference implementation of the matching criterion (Section 4.2), we found a
 392 bug in the function that implements it (get_counts_for_protein, in the DeepPeptide codebase),
 393 reproduced below in simplified form:

```

394 for idx, row in true_df.iterrows(): # idx: true-segment index
395     true_start, true_stop = row['start'], row['stop']
396     for idx, row in pred_df.iterrows(): # idx overwritten with pred index
397         pred_start, pred_stop = row['start'], row['stop']
398         ...
399         if start_match and stop_match:
400             true_df.loc[idx, 'matched'] = True # BUG: idx is now the
401             pred_df.loc[idx, 'matched'] = True # predicted index, not the true one
402         break
403

```

The outer loop iterates over true segments and the inner loop over predicted segments, but both reuse the loop variable `idx`. By the time a match is found, `idx` refers to the predicted segment, so `true_df.loc[idx, 'matched']` marks the wrong true segment. The effect is not large on average (it is partly self-correcting), but it costs a true positive whenever it fires, so both recall and precision come out low, and F1 with them. Across the nested-CV grid the correction raises recall by 0.023 to 0.029 and precision by 0.007 to 0.010. The loss concentrates in proteins where the model predicts more segments than the protein actually has. This bug is present in the upstream DeepPeptide code, not something we introduced.

We did not silently patch this function in place, since that would shift absolute numbers without documenting it. Instead we implemented a separate, corrected matcher and used it to re-score the results in this paper: the earlier single-split runs reported in Appendix D, except two whose model classes no longer exist in the code and whose checkpoints cannot be rebuilt, and the full 5×4 nested-cross-validation grid in Table 1 (all 140 cells of the seven configurations, and the 20 further cells of the 2022-release reproduction), by re-running test-partition inference from each cell’s saved checkpoint on the machine that held them. Every table in the main text uses the corrected matcher; the original (buggy) numbers are kept alongside in the released run artifacts for direct comparison with the published methodology, not reproduced here. Checkpoint selection inside each cell ran against the original matcher, as the training loop was written before the audit, so the selected epoch is the one that criterion preferred; the reported scores are the corrected matcher’s reading of that checkpoint.

Re-scoring the grid also recomputed the original, buggy matcher on the same decoded segments, so that each cell could be checked against the number already published for it. Over the 80 cells for which that check could be run, the two agree to 1.2×10^{-3} F1 at worst and to 3×10^{-7} at the median, the worst case being one cell of the 2022-release reproduction and the next largest 5×10^{-4} . What remains is inference nondeterminism rather than disagreement between the matchers, which are deterministic functions of the same decoded segments.

Across the eight runs the correction shifts mean F1 by +0.015 to +0.022, always upward, since the bug was discarding true positives. On ESM-2 it leaves the ordering of the four configurations intact: base < boundary head < adapter < combined, before and after correction. That ordering is not robust, and we would not build an argument on it. The boundary-head/adaptor gap is 0.005 against a fold-level standard deviation of 0.026; per outer fold the full ordering holds in four folds of five after correction and three of five before it, and on peptides alone it holds in one fold of five. On ESM-C 6B the three configurations we have order the same way, base < boundary head < combined.

D Earlier single-split protocol: unconfirmed observations

This appendix reports findings from an earlier stage of the project, obtained under a single-split protocol that predates the 5×4 nested cross-validation adopted for the confirmed results in Section 5. We report them here, explicitly separated from the main results, both because they motivated the switch to nested CV and because several of them describe architectural or data ideas that were never re-tested under the more rigorous protocol. None of the numbers below should be read with the same confidence as Table 1.

D.1 Protocol

Data was split into seven GraphPart folds (30% identity threshold, motif-balanced as in Section 4.1), with folds assigned fixed roles: four folds for training, one for validation (epoch selection), one for model selection (comparing architectures), and one originally intended as a fully sealed test

fold. This separated architecture selection from evaluation, but the evaluation itself was still a single draw: one random assignment of roles, one pair of held-out folds. Several modifications changed the *sign* of their measured effect between the two held-out folds. Replacing ESM-2 with ESM-C 6B measured +0.074 on one and −0.011 on the other; ESM-C 6B against ESM-C 600M gave +0.028 and −0.021; the net effect of the 3Di channel gave +0.015 and −0.018. Effects that kept their sign still moved by a lot: the isolated gated adapter measured −0.001 on one fold and +0.045 on the other. Figure 4 shows one reason, namely that the folds differ substantially in segment-length composition. The two held-out folds turn out to be complementary in segment-length composition, fold 2 concentrated at 10–24 residues and fold 5 bimodal, so results below pool them (model-select $\cup$ sealed, $\approx 2,300$ proteins) to reduce (but not eliminate) this instability; this pooling is why the “sealed” fold is not, in practice, a held-out test set independent of model selection.

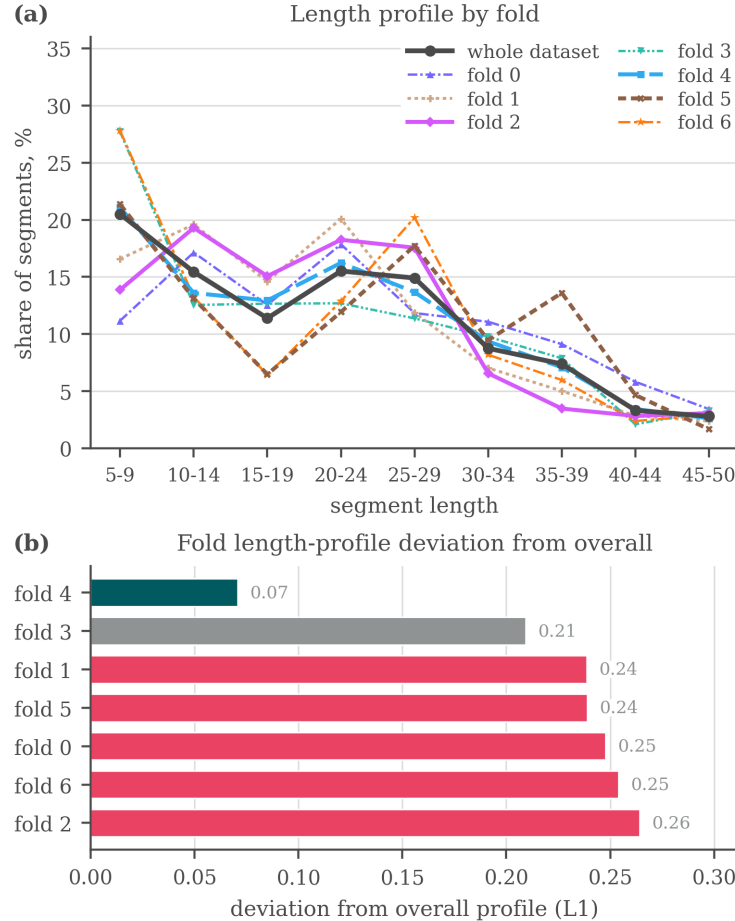


Figure 4: Why the folds of the earlier seven-fold split are not interchangeable: (a) the segment-length profile of each fold against the profile of the whole dataset, (b) the L_1 distance between the two. Fold 4 tracks the overall distribution closely (0.07) while five of the seven folds sit above 0.23. Balancing was done on cleavage motif and homology, not on segment length, so this axis was left free. The same holds for the five-fold split used in the main text, where the imbalance is taxonomic (Table 2).

460 D.2 Summary of tested modifications

461 Table 4 summarizes the modifications tested under this protocol, with the corrected matcher (Ap-
 462 pendix C) applied throughout. Three of these rows were later re-tested under nested cross-validation:
 463 the boundary head, the adapter (Section 3), and the ESM-C 6B embedding swap. Two survived. The
 464 embedding swap did not: nested cross-validation puts ESM-C 6B at 0.588 ± 0.016 against ESM-2’s

0.573 $\pm$ 0.025 (Table 1), overlapping intervals, where this table records a confident +0.03. Everything else below remains at the confidence of a single pooled split.

Table 4: Modifications tested under the earlier single-split protocol, with their measured effect on F1 (pooled held-out folds) and an informal verdict. None of these effect sizes should be compared directly to the nested-CV numbers in Table 1.

Modification	Type	Measured effect on F1	Verdict
ESM-C 6B instead of ESM-2	embedding swap	+0.03, unstable across folds	helps
Boundary head on ESM-C 6B	decoder addition	+0.05, +0.07, consistent	helps
Boundary head on ESM-2	decoder addition	≈ 0 (CI includes zero)	no effect
Structural channel (3Di)	extra input	propep. +0.02 / pep. -0.03, net trade-off	no effect
ESM-C 6B compression 2560 $\rightarrow$ 256	optimization	≈ 0 , 10 $\times$ narrower input and 16 $\times$ fewer trainable parameters	no effect
Gated adapter on ESM-C 6B (pLM re-projection)	input adapter	+0.022, CI [+0.008, +0.038]	helps
Bond-prediction auxiliary loss (on ESM-C 6B)	extra loss	pep. -0.05, net -0.015; neutral on ESM-2	harmful
Telescopic segment CRF	decoder addition	≈ 0 on ESM-2, +0.022 on ESM-C 6B, no CI available	unresolved

Note the apparent tension with Table 1: under this earlier protocol, the boundary head on ESM-2 looked like it had no effect, while nested cross-validation resolved a confirmed +0.024 F1 gain for the same modification on the same embedding. We read this as evidence that the single-split protocol lacked the statistical power to resolve an effect of this size, not as a contradiction – and as the clearest illustration of why we do not treat single-split numbers as findings in this paper. Whether the boundary head’s effect is specifically larger on richer embeddings (as the ESM-C 6B rows above suggest) is accordingly left as an open question, not stated as a conclusion here.

D.3 Additional exploratory modifications

A larger set of ideas was tried under the same single-split protocol and did not show a clear, consistent benefit; we list them for completeness and as candidates for future re-testing, without effect-size claims:

- **Known-peptide dictionary.** An Aho–Corasick index of peptides from the training data was used to flag substring matches in a query sequence, injected as a bonus on the CRF state emissions, as a bias on the start, inside and end emissions specifically, fused into the encoder hidden state, and concatenated with the embedding before the encoder. No consistent benefit was observed, and the checkpoint for one of the variants can no longer be loaded.
- **Structural features.** Features extracted from predicted 3D structures (AlphaFold2 representations via the AFToolkit codebase, with several confidence-based filtering variants) and from the ProstT5 [Heinzinger et al., 2024] structural alphabet (3Di, as used in Foldseek [van Kempen et al., 2024]) were tested as additional input channels; results were mixed (Table 4).
- **LoRA fine-tuning.** Low-rank adaptation of the last few pLM layers, instead of fully frozen embeddings, scored 0.558 and 0.567 against 0.621 for the frozen baseline, all three on the 2022 release under its own five-fold split rather than the protocol of Appendix D. It also ran for fewer epochs in the same time budget, though we did not measure the cost directly.
- **Projector variants.** Multi-scale and multi-branch projectors between the embedding and the CNN–BiLSTM were tried as alternative re-projection schemes to the adapter of Section 3.
- **Structural-projection width and telescopic CRF.** The width of the structural-feature projection (16/32/48 units) left pooled F1 flat at 0.691, 0.697 and 0.693, the telescopic segment

CRF, an alternative decoder formulation that scores whole segments through a relative-position head, came out level with the base decoder on ESM-2 and +0.022 ahead on ESM-C 6B. Per-protein outputs were not retained for either run, so neither number carries a confidence interval and we treat the modification as untested rather than as having no effect.

None of these modifications showed a consistent gain large enough, under the single-split protocol, to justify testing under nested cross-validation ahead of the two reported in the main text. The complete experiment log (including runs not summarized above) is maintained in the project repository rather than reproduced here, since it was collected under a protocol we no longer treat as sufficient evidence on its own.

D.4 Complete tolerance sweep

Table 5 repeats the tolerance sweep of Table 1 with the fold-level standard deviation on every cell. The paired localization comparison of Section 5 runs on the true segments that a variant and the base both place within ± 3 : 27,013 segments for the boundary head, 29,277 for the adapter and 29,394 for the two together, pooled over the twenty cells of each configuration.

Table 5: Segment F1 by boundary-match tolerance, 5×4 nested cross-validation, corrected matcher, mean over the five outer folds.

Configuration	± 3	± 2	± 1	exact	gap growth
ESM-2, base	0.573 ± 0.025	0.540 ± 0.026	0.461 ± 0.016	0.345 ± 0.008	+0.000
ESM-2 + boundary head	0.597 ± 0.026	0.566 ± 0.024	0.488 ± 0.015	0.384 ± 0.015	+0.015
ESM-2 + adapter	0.602 ± 0.026	0.572 ± 0.024	0.491 ± 0.011	0.396 ± 0.020	+0.022
ESM-2 + boundary head + adapter	0.630 ± 0.021	0.599 ± 0.023	0.518 ± 0.016	0.432 ± 0.013	+0.029

E Segment quality without a tolerance

The ± 3 criterion of Section 4.2 is the benchmark’s, and Section 1 says why it cannot see the improvement this paper is about. This appendix scores the same predictions without it. Predicted and true segments of the same type are matched greedily one to one by intersection over union, and a match counts as a hit when the overlap clears a threshold τ ; precision and recall use all predicted and all true segments as their denominators, so nothing is conditioned on what either model happened to find. Every configuration is scored on the 35,504 protein-cells common to all runs, which takes the coverage difference between the two embeddings (Section 4.1) out of the comparison.

Table 6: Segment F1 under overlap matching. The first column requires only that a predicted and a true segment of the same type share one residue; the rest require an intersection over union of at least τ . Same cells as Table 1, restricted to the protein set common to every run; mean over the five outer folds.

	overlap	$\tau \geq 0.5$	$\tau \geq 0.8$	$\tau \geq 0.9$	$\tau \geq 0.95$
<i>ESM-2</i>					
base	0.729	0.667	0.569	0.487	0.411
+ boundary head	0.715	0.668	0.585	0.515	0.438
+ adapter	0.734	0.686	0.590	0.516	0.441
+ both	0.746	0.703	0.616	0.546	0.474
<i>ESM-C 6B</i>					
base	0.738	0.672	0.574	0.496	0.419
+ boundary head	0.735	0.704	0.624	0.555	0.478
+ both	0.759	0.727	0.649	0.582	0.507

Placement, not coverage. The looser the criterion, the less either addition is worth. Paired by outer fold against the ESM-2 base, at a gate that asks only for a single residue of overlap the boundary head is worth -0.013 ± 0.026 and the adapter $+0.006 \pm 0.012$, neither distinguishable from zero; at $\tau \geq 0.5$ they are worth $+0.001 \pm 0.021$ and $+0.019 \pm 0.017$, and at $\tau \geq 0.9$ $+0.028 \pm 0.004$ and $+0.029 \pm 0.015$, both on five folds of five. Underneath that near-zero F1 at the overlap gate the two

move in opposite directions: the head issues 219 fewer segments per fold, buying $+0.022$ precision for -0.035 recall, while the adapter issues 130 more and buys $+0.024$ recall for -0.021 precision. Neither finds appreciably more of the annotation than the base architecture does; what changes is where the boundaries land. This is the same conclusion Table 1 reaches through a tolerance, reached without one.

The mean IoU taken over *all* true segments moves the other way for the head, 0.576 to 0.559, while its F1 rises at every threshold. That is the precision effect of Table 1 seen from another angle: the head withdraws weak predictions, so some true segments lose a poor match altogether, and the ones that keep a match are placed better.

Which end moves. Scoring cleavage sites rather than segments, at exact placement and against all annotated sites, separates the two additions again. The boundary head improves the C-side of a segment, $+0.028 \pm 0.006$ on five folds of five, and leaves the N-side alone, -0.007 ± 0.020 on two of five. The adapter improves both, $+0.022 \pm 0.006$ and $+0.028 \pm 0.009$, and the two together improve both further, $+0.042 \pm 0.010$ and $+0.060 \pm 0.006$. Split by segment type, the head's C-side gain is largest on propeptides ($+0.037 \pm 0.021$, five of five), whose C-terminal cut is where the mature peptide begins and is also the base model's weakest site: it scores 0.476 there against 0.525 on peptide C-sides.

E.1 Supplementary figures

The figures below are from the same earlier development stage as Table 4 and are included for completeness; none of them should be read as confirmed evidence in the sense of Section 5.

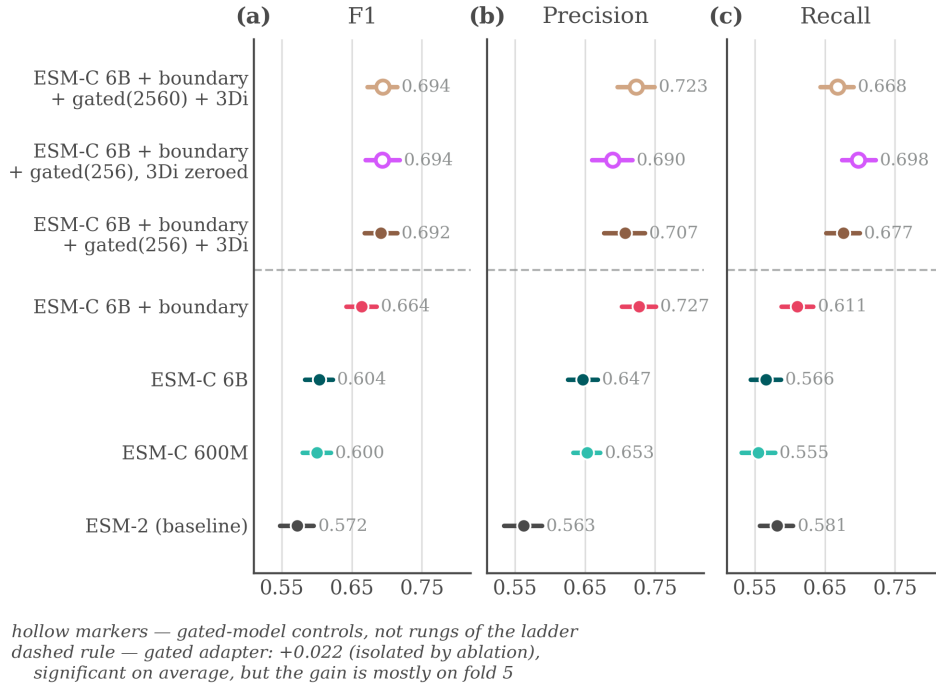


Figure 5: Single-split comparison on the pooled held-out folds at ± 3 : F1, precision and recall with bootstrap confidence intervals over 2,322 proteins. The top row is a gated-projector control at full width, not the configuration carried into the main text.

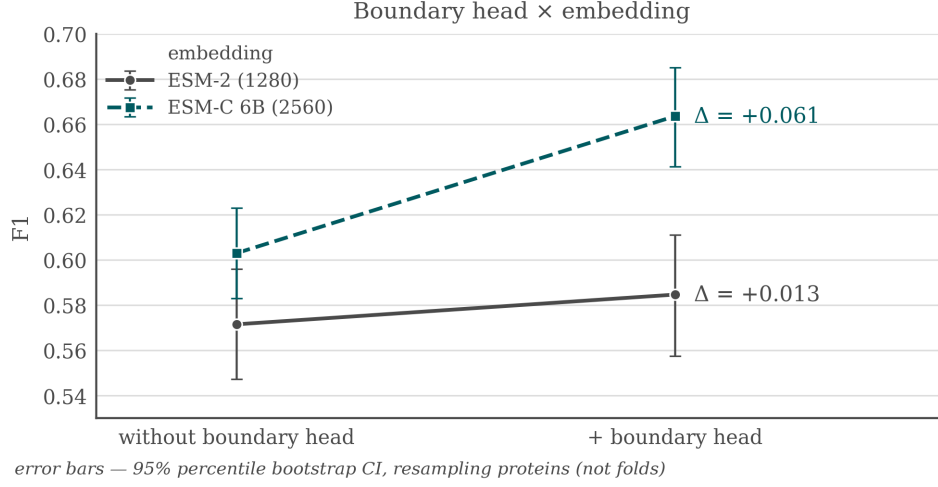


Figure 6: Boundary head × embedding interaction under the single-split protocol: F1 gain from adding the boundary head on top of ESM-2 vs. on top of ESM-C 6B. This is the earlier, unconfirmed version of the interaction question raised in Section 5.

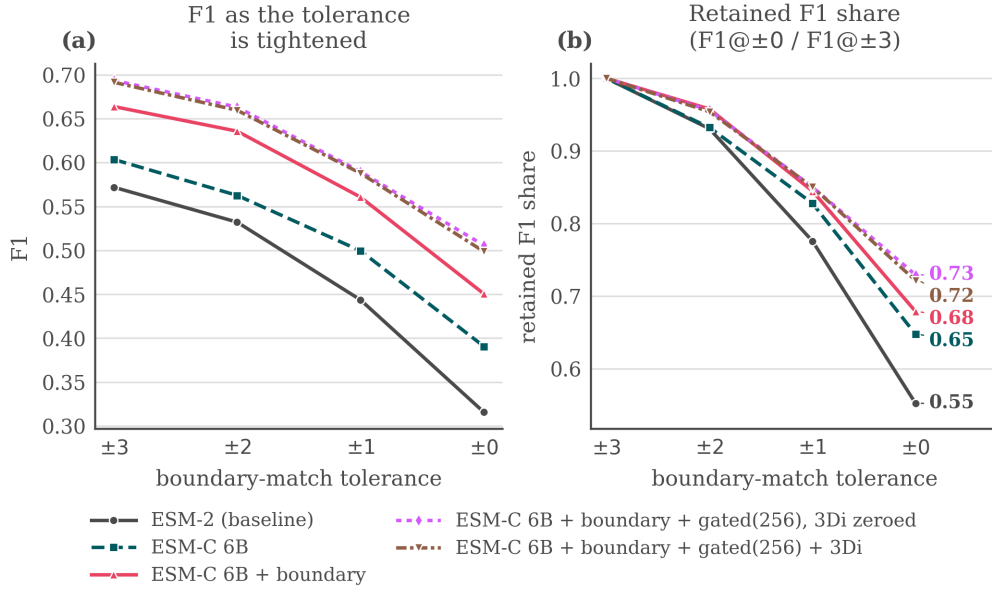


Figure 7: Quality as the matching tolerance is tightened from ± 3 to an exact match (± 0), single-split protocol, pooled held-out folds.

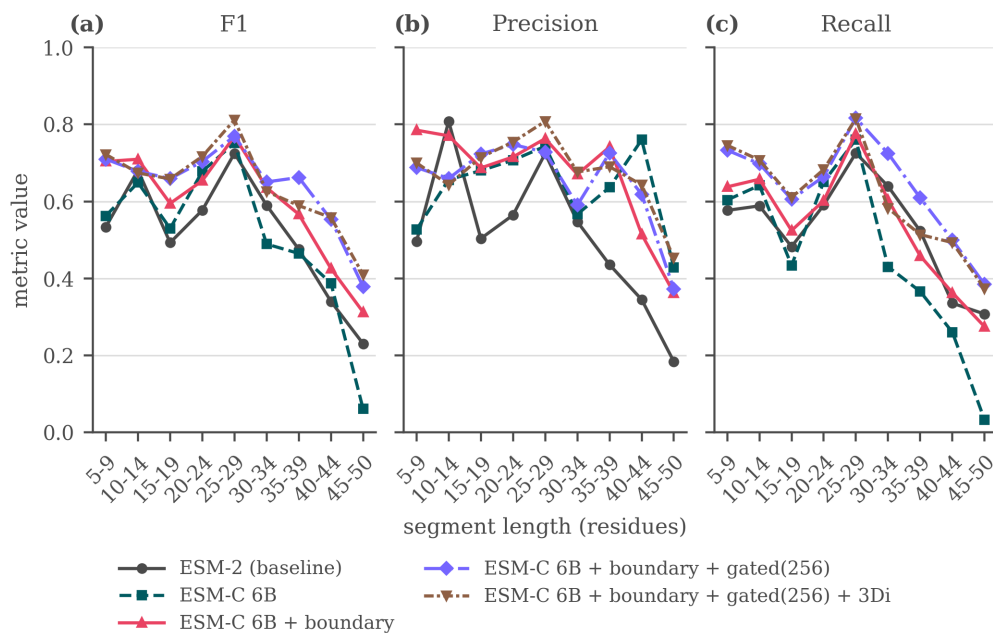


Figure 8: F1, precision, and recall as a function of segment length, single-split protocol, pooled held-out folds.

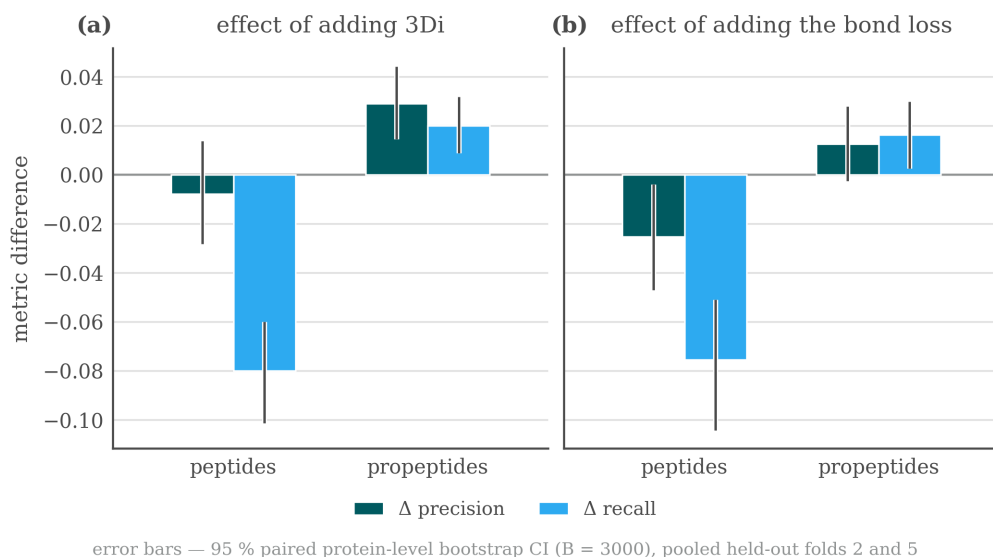


Figure 9: Precision/recall trade-offs by segment type when adding the 3Di channel or the bond-prediction loss, single-split protocol, pooled held-out folds.

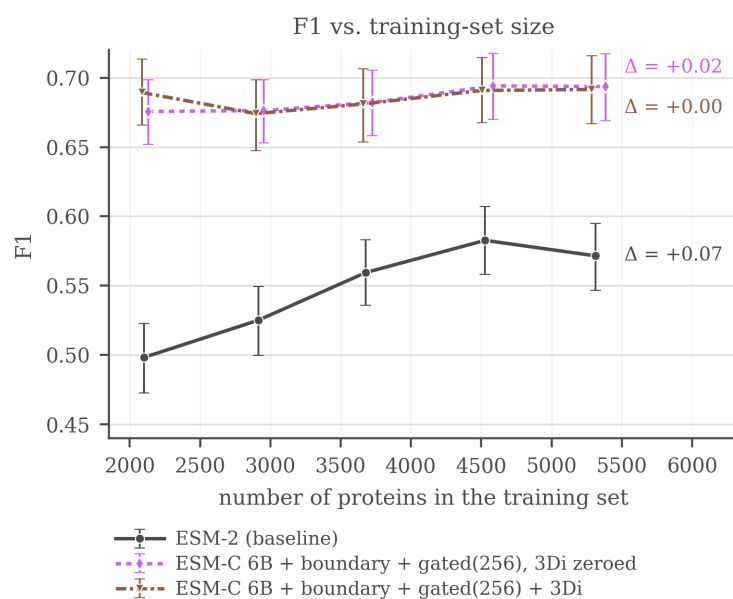


Figure 10: F1 (± 3) against the number of training proteins, single-split protocol. ESM-2 rises from 0.498 at 40% of the training folds to 0.583 at 85%, then falls back to 0.572 at 100%, so it is still gaining over most of the range but not at the last point. The ESM-C 6B curves are flat from the smallest size tested, with movement smaller than the bootstrap interval, so nothing is observed saturating: they simply never climb.

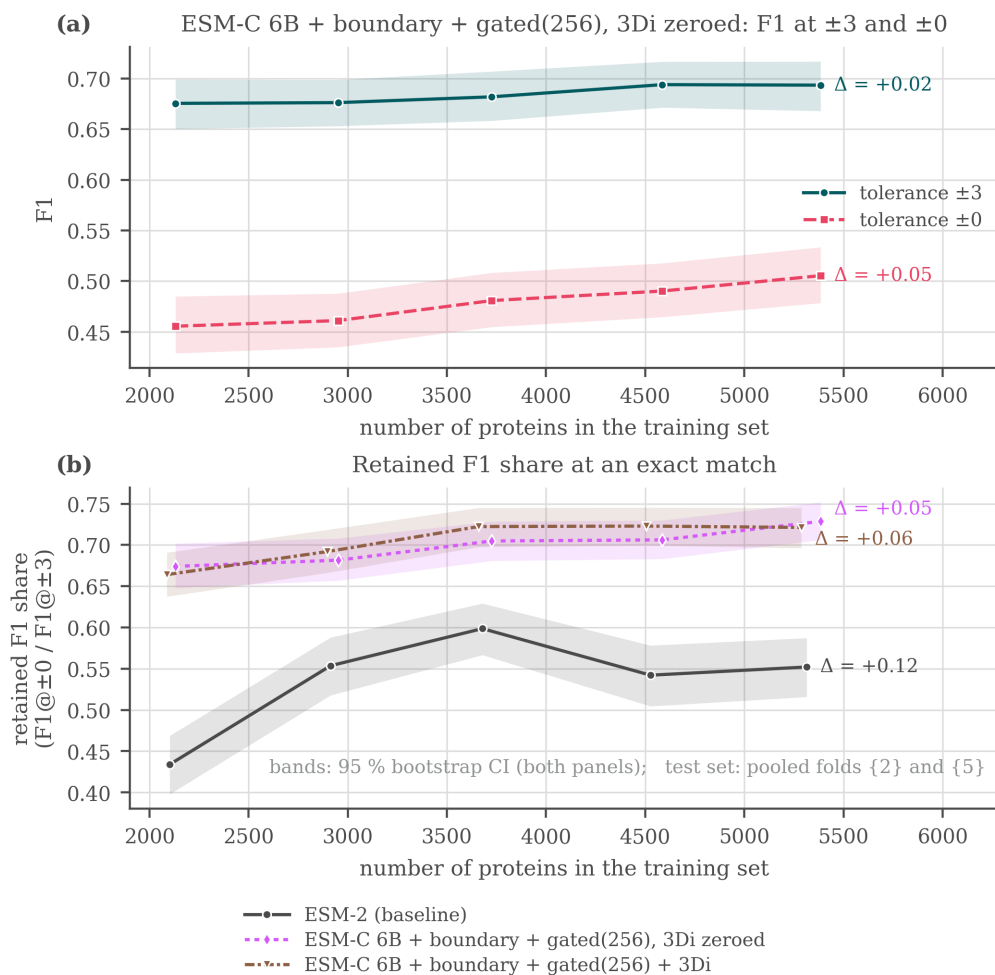


Figure 11: F1 against training-data volume at two tolerances, single-split protocol. The ± 3 curve is flat for the strong model (0.676 to 0.694, overlapping intervals) while the exact-match curve rises monotonically (0.455 to 0.505), though its endpoint intervals still overlap slightly. Finding a peptide saturates with data before placing its ends does.

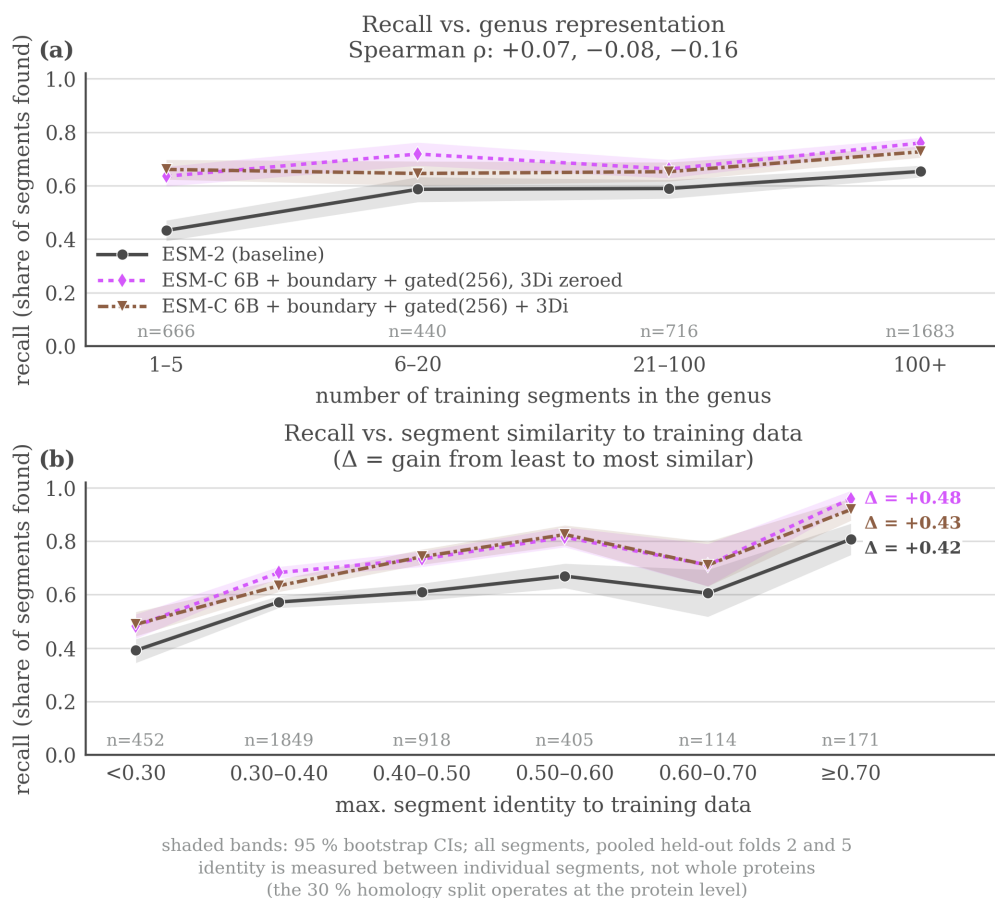


Figure 12: Recall on held-out peptides as a function of (a) how well-represented the protein’s genus is in training, and (b) maximum sequence identity to a training segment, single-split protocol. For the two ESM-C models recall tracks maximum identity to a training segment far more than genus abundance; for the ESM-2 baseline the two axes are closer to comparable. The x axis in (a) counts training *segments* in the genus, not proteins.